SUSTENTACIÓN DE AUTOMATIZACIÓN DE PRUEBAS

Automatización de Pruebas con PHPUnit 10.5
Facturación Electrónica SUNAT UBL 2.1

Arquitectura de testing, ciclo de vida de pruebas, atributos de PHP 8, aserciones estrictas, simulación con Mocks y certificación del 100% de la suite tributaria.

INTEGRANTES DEL EQUIPO DE QA:

- Apolinario Rodriguez, Luis Fernando
- Umeres Vilca, Gustavo Aron
- Vasquez Falcón, Mariana Ysabel
- Ventocilla Alcantara, Alessandro Tito

MÉTRICA DE LA SUITE:

39 Tests • 140 Aserciones

100% OK • 0.21 segundos en RAM (:memory:)

FUNDAMENTOS DEL FRAMEWORK

1. ¿Qué es PHPUnit y por qué lo usamos?

PHPUnit 10.5.66

🏆 El Estándar de Facto en PHP

Creado por **Sebastian Bergmann**, PHPUnit es la herramienta indiscutible para pruebas unitarias e integración en el ecosistema profesional de PHP.

- **PHPUnit 10:** Salto generacional que elimina opciones obsoletas e introduce tipado estricto sin tolerancia a advertencias.
- **Alineado a PHP 8.2+:** Soporte nativo para atributos de lenguaje en reemplazo de anotaciones dentro de bloques de comentarios.
- **Totalmente Aislado:** No requiere servidores web, ni navegadores, ni bases de datos físicas para probar la lógica central.

⚙️ Orquestación y Jerarquía de Suites

A través de backend/phpunit.xml, se organiza la ejecución bajo la Pirámide de Testing:

- **Suite Unit:** Pruebas matemáticas puras en memoria (microsegundos).
- **Suite Integration:** Pruebas de esquemas XML UBL 2.1 y persistencia atómica SQLite.
- **Suite Feature:** Casos de uso de negocio con Mocks de SUNAT.
- **Reporte Testdox:** Genera documentación legible para humanos a partir del nombre de cada método.

2. El Ciclo de Vida de las Pruebas (Test Lifecycle)

Aislamiento Total

🔄 Los Hooks de Inicialización

Para evitar **pruebas intermitentes o contaminadas (Flaky Tests)**, PHPUnit proporciona métodos de enganche que garantizan un entorno limpio:

- **setUpBeforeClass():** 1 vez antes de toda la clase.
- **setUp():** Antes de *cada prueba individual*. Reinicia la BD en memoria.
- **tearDown():** Después de *cada prueba*. Destruye objetos en RAM.
- **tearDownAfterClass():** Cierre final de recursos.

¿Cómo lo aplicamos en nuestro sistema?

Cada test crea una conexión nueva a sqlite::memory: y siembra el catálogo de 9 productos. Ningún test hereda datos del test anterior.

SqlitePersistenceIntegrationTest.php

```php
protected function setUp(): void
{
    parent::setUp();

    // 1. Instanciamos SQLite puro en RAM (0 residuos en disco)
    $this->pdo = new PDO('sqlite::memory:');
    $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 2. Migramos tablas y sembramos los 9 productos
    $this->migrador->ejecutar($this->pdo);
    $this->seeder->sembrarCatalogo($this->pdo);
}

protected function tearDown(): void
{
    $this->pdo = null; // Destruye la BD en RAM al instante
    parent::tearDown();
}
```

3. El Patrón AAA (Arrange - Act - Assert) & Principios FIRST

Estándar de Calidad

🧩 Las Tres Fases de una Prueba Limpia (AAA)

1. Arrange (Preparar): Configurar el estado previo, crear entidades de dominio (RUC, ítems) y preparar dobles de prueba (Mocks).

2. Act (Actuar): Ejecutar el método o caso de uso bajo prueba (ej. calcular() o emitirFactura()).

3. Assert (Afirmar): Verificar que el resultado obtenido coincide exactamente con la expectativa técnica o fiscal.

⚡ Los 5 Principios FIRST de Robert C. Martin

- **Fast (Rápidas):** 39 tests corren en **0.21 segundos**. Si fueran lentas, los desarrolladores no las correrían.
- **Independent (Independientes):** Ningún test depende del resultado de otro. Pueden correr en cualquier orden.
- **Repeatable (Repetibles):** Dan exactamente el mismo resultado en local, en servidor o en CI/CD.
- **Self-validating (Auto-evaluables):** El resultado es binario: verde (pasa) o rojo (falla). Cero interpretación manual.
- **Timely (Oportunas):** Se diseñan junto con la lógica de negocio.

4. Pruebas Parametrizadas: Atributo #[DataProvider]

PHP 8 Native Attributes

🎯 Cero Código Repetitivo (DRY)

En lugar de escribir 10 métodos de prueba distintos para validar 10 números de RUC diferentes, el atributo #[DataProvider] inyecta una matriz de tuplas directamente al método de prueba.

- **Equivalencia en Java:** Es exactamente el equivalente a @ParameterizedTest y @CsvSource de JUnit 5.
- **Técnica de QA:** Aplica *Partición de Equivalencia*: RUCs 20 (empresas), RUCs 10 (personas naturales) y RUCs 15 (FFAA).

ValueObjectsTest.php

```php
#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidos(string $ruc, string $tipo): void
{
    // Arrange & Act: Instanciamos el Value Object
    $vo = new Ruc($ruc);

    // Assert: Verificamos identidad estricta
    $this->assertSame($ruc, $vo->valor());
}

public static function proveedorRucsValidos(): array
{
    return [
        'RUC 20 SUNAT'    => ['20131312955', 'Jurídica'],
        'RUC 20 BCP'      => ['20100047218', 'Bancaria'],
        'RUC 20 ALICORP'  => ['20100055237', 'Industrial'],
        'RUC 10 PERSONA'  => ['10456789124', 'Natural'],
    ];
}
```

5. Catálogo de Aserciones: assertSame vs assertEquals

Tipado Estricto

⚠ El Peligro de la Igualdad Débil (assertEquals)

En PHP, assertEquals usa coerción débil (==). Por ejemplo, "100" == 100 da VERDADERO. En sistemas financieros, esto puede esconder errores de redondeo o conversiones accidentales a string.

🛡 Identidad Estricta (assertSame)

Comprueba valor y tipo (===). Si esperas float 180.00 y el motor devuelve int 180, la prueba falla legítimamente.

💥 Aserciones de Excepción

expectException() permite auditar los caminos de fallo: si el usuario intenta violar una norma SUNAT, el test valida que se lance la excepción correcta.

ComprobanteDomainRulesTest.php

```php
// Regla SUNAT 2017: Factura exige RUC; rechaza DNI
public function testImpideEmitirFacturaAClienteConDni(): void
{
    // Assert previo: Esperamos la excepcion tributaria
    $this->expectException(DomainValidationException::class);
    $this->expectExceptionMessage(
        "Solo se emiten Facturas (01) a clientes con RUC"
    );

    // Arrange
    $emisor = new Emisor(new Ruc('20131312955'));
    $clienteConDni = new Cliente(new Dni('72345678'));

    // Act: Dispara la excepcion interceptada
    Comprobante::crearFactura(
        'F001', 1, $emisor, $clienteConDni
    );
}
```

6. Dobles de Prueba: Mocks y Spies de SUNAT (Martin Fowler)

Simulación Resiliente

🎯 ¿Por qué Mocks y no el Web Service Real?

- **Inestabilidad:** Los servidores SOAP de SUNAT sufren micro-cortes y lentitud (2 a 5 seg).
- **Costo y Certificados:** Requiere certificados digitales y conexión permanente a internet.
- **Escenarios Catastróficos:** Con un Mock podemos forzar caídas HTTP 504 o rechazos raros (ej. CDR 2800) a voluntad.

🕵️ Verificación de Comportamiento con Spies

Con expects($this->never()) garantizamos que si una regla de negocio falla (ej. sin stock), el sistema **jamás gaste un correlativo ni llame a la red.**

FacturacionElectronicaMocksFeatureTest.php

```php
// Spy: Verifica que NUNCA se llame a SUNAT si no hay stock
public function testNoLlamaASunatSiNoHayStock(): void
{
    // 1. Creamos el Mock como Spy
    $sunatSpy = $this->createMock(SunatGatewayInterface::class);

    // Asercion de comportamiento: NUNCA debe invocarse
    $sunatSpy->expects($this->never())
             ->method('enviarComprobante');

    $this->expectException(StockInsuficienteException::class);

    // 2. Intenta vender 50 laptops (solo hay 15 en stock)
    $service = new FacturacionService($this->kardex, $sunatSpy);
    $service->emitirFactura($pedidoDe50Laptops);
}
```

ESTRATEGIA DE CALIDAD

7. Pirámide de Pruebas de Mike Cohn Aplicada al Sistema

Distribución Óptima

Base: 26 Pruebas Unitarias (67%) — 0.05 segundos

Prueban Value Objects (RUC Módulo 11, DNI), Calculadora Tributaria (IGV 18%, ICBPER, SPOT) y Reglas de Comprobante en memoria pura.

Medio: 7 Pruebas de Integración (18%) — 0.08 segundos

Validan la estructura XML UBL 2.1 con XPath, la firma digital XMLDSIG SHA-256, persistencia SQLite e idempotencia.

Cúspide: 6 Pruebas Feature (15%) — 0.08 segundos

Simulan el ciclo de venta completo, caídas de red, notas de crédito y reversión de Kardex con Mocks de SUNAT.

MÉTRICAS CONSOLIDADAS DE LA SUITE:

39 / 39

Pruebas Exitosas (100% OK)

140	0.21s
Aserciones Auditadas	Tiempo de Ejecución

8. Algoritmo SUNAT Módulo 11 en Value Object Ruc

Ponderación Oficial SUNAT

📐 Algoritmo Ponderado de 11 Dígitos

SUNAT no valida el RUC únicamente por su longitud, sino aplicando una fórmula matemática ponderada sobre los primeros 10 dígitos:

```
Factores: [5, 4, 3, 2, 7, 6, 5, 4, 3, 2]
```

- $S = \sum_{i=0}^{9} (d_i imes f_i)$
- $Resto = 11 - (S \pmod{11})$
- Si $Resto == 10 \implies 0$; Si $Resto == 11 \implies 1$.
- El dígito verificador debe coincidir con el dígito 11.

Beneficio en QA:
Rechaza RUCs falsificados o errores tipográficos antes de registrar datos o enviar facturas a SUNAT.

src/Domain/ValueObjects/Ruc.php

```php
private function validarModulo11(string $ruc): void
{
    $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
    $suma = 0;
    for ($i = 0; $i < 10; $i++) {
        $suma += ((int) $ruc[$i]) * $factores[$i];
    }

    $resto = 11 - ($suma % 11);
    $dvEsperado = match ($resto) {
        10 => 0,
        11 => 1,
        default => $resto
    };

    if ($dvEsperado !== (int) $ruc[10]) {
        throw new DomainValidationException(
            "Dígito verificador de RUC inválido (esperado: $dvEsperado)"
        );
    }
}
```

9. Liquidación de Comprobante Mixto Complejo

Convivencia Multirégimen

⚖️ Afectaciones Tributarias SUNAT

- **Gravado (10):** Aplica el 18% de IGV (Base S/ 1,000 $\implies$ S/ 180 IGV).
- **Exonerado (20):** Libros Ley 31053 (0% IGV).
- **Gratuito (15):** Muestras promocionales. Tiene IGV referencial pero **no suma al total a pagar.**
- **Bolsas Plásticas:** Impuesto ICBPER fijo (S/ 0.50 cada una) sin contaminar la base del IGV.

Verificación Matemática: El test comprueba que cada bolsa fiscal se totalice de forma estricta y sin pérdidas de centavos.

CalculadoraTributariaTest.php

```php
public function testCalculaComprobanteMixtoComplejo(): void
{
    $calc = new CalculadoraTributaria();
    $totales = $calc->calcular([
        new Item('Monitor LED', 1000.0, 1, '10'), // Gravado 18%
        new Item('Libro Java',  200.0,  1, '20'), // Exonerado 0%
        new Item('Regalo',      50.0,   1, '15'), // Gratuito
        new Item('Bolsa',       0.10,   2, '10', icbper: true)
    ]);

    $this->assertSame(1000.0, $totales->gravado);
    $this->assertSame(180.0,  $totales->igv);
    $this->assertSame(200.0,  $totales->exonerado);
    $this->assertSame(1.00,   $totales->icbper); // 2 bolsas x S/ 0.50
    $this->assertSame(1381.20, $totales->totalPagar);
}
```

10. Análisis de Valores Límite: Boleta Mayor a S/ 700.00

Boundary Value Analysis

🔍 Técnica de Caja Negra: Valores Límite

Los defectos de software se concentran en las fronteras de las condiciones lógicas. La **Resolución SUNAT 007-99** establece que una boleta anónima sólo puede emitirse hasta S/ 700.00.

Frontera Válida:
Monto = S/ 700.00 $\implies$ Pasa sin DNI (Anónimo).

Frontera Inválida:
Monto = S/ 700.01 $\implies$ Lanza Error 2511 exigiendo DNI.

ComprobanteDomainRulesTest.php

```php
// Test 22: Boleta > S/ 700 sin DNI DEBE ser rechazada
public function testImpideBoletaMayorASetecientosSinDni(): void
{
    $this->expectException(DomainValidationException::class);
    $this->expectExceptionMessage(
        "Boletas mayores a S/ 700 requieren identificar al cliente (DNI)"
    );

    // Intentamos emitir S/ 767.00 a 'Clientes Varios' (sin DNI)
    Comprobante::crearBoleta(
        'B001', 1, $this->emisor, Cliente::clientesVarios(),
        [new Item('Televisor', 650.0, 1, '10')] // + IGV = 767.00
    );
}
```

11. Prueba de Seguridad: Tamper Test (Código 2335)

Ataque de Alteración de Importe

¿Qué pasa si un usuario malicioso intercepta el archivo XML ya firmado digitalmente y cambia el monto de **S/ 1,180.00** a **S/ 10.00** para evadir impuestos?

- El algoritmo SHA-256 recalcula el digest del árbol XML canónico.
- Al detectar que el valor no coincide con el nodo <ds:DigestValue>, SUNAT rechaza la transacción.
- **Código Fiscal Oficial:** Rechazo **2335** - **La firma digital ha sido alterada.**

XmlAndCryptoIntegrationTest.php

```php
public function testDetectaAdulteracionDeImporte(): void
{
    $xmlFirmado = $this->firmador->firmar($xmlOriginal);

    // ATAQUE: Alteramos el importe ya timbrado
    $xmlAdulterado = str_replace(
        '1180.00',
        '10.00',
        $xmlFirmado
    );

    // Evaluamos la respuesta del simulador de SUNAT
    $cdr = $this->sunatSimulator->validarXml($xmlAdulterado);

    $this->assertFalse($cdr->esAceptado());
    $this->assertSame(2335, $cdr->codigoRespuesta);
    $this->assertStringContainsString(
        'El digest no coincide', $cdr->observaciones
    );
}
```

12. Prevención de Doble Emisión por Concurrencia

Anti-Doble Clic

⚡ El Escenario del Doble Clic del Cajero

Si la cajera da doble clic por error al botón "Emitir Factura" o si la conexión parpadea:

- Un sistema sin idempotencia crearía dos facturas con números distintos y cobraría el doble.
- Nuestro sistema usa la tabla idempotency_keys con ciclo:
  IN_PROGRESS → COMPLETED
- El segundo intento detecta la clave y **retorna la respuesta original en caché** sin crear comprobantes duplicados.

SqlitePersistenceIntegrationTest.php

```php
public function testTablaIdempotenciaPrevieneDobleEmision(): void
{
    $key = 'uuid-cajero-terminal-01';

    // Intento 1: Emisión exitosa
    $res1 = $this->service->emitirConIdempotencia($key, $orden);

    // Intento 2: Inmediato con la misma clave UUID
    $res2 = $this->service->emitirConIdempotencia($key, $orden);

    // Las respuestas son identicas
    $this->assertSame($res1->correlativo, $res2->correlativo);

    // La base de datos solo almaceno 1 comprobante
    $count = $this->db->query("SELECT count(*) FROM comprobantes")->fetchColumn();
    $this->assertSame(1, (int) $count);
}
```

13. Resiliencia: Rollback Automático de Kardex ante Rechazo

Invariante Transaccional

📦 Protección de Stock ante SUNAT

Al emitir una factura, el sistema reserva el inventario. Pero ¿qué ocurre si SUNAT responde que el RUC del cliente está **de baja de oficio (Código 2800)**?

- La factura no puede tener validez fiscal.
- El caso de uso debe realizar un **Rollback atómico** en el Kardex.
- Las 15 laptops reservadas regresan de inmediato al almacén, evitando pérdidas de mercadería.

FacturacionElectronicaMocksFeatureTest.php

```php
public function testRevierteElStockAutomaticamenteSiSunatRechaza(): void
{
    // Mock de SUNAT simulando Rechazo 2800 (RUC de baja)
    $this->sunatMock->method('enviarComprobante')
                    ->willReturn(new CdrResponse(codigo: 2800));

    // Stock inicial: 15 laptops
    $this->assertSame(15, $this->kardex->obtenerStock('LAPT-LEN-01'));

    // Ejecutamos la emision (falla en SUNAT)
    $this->service->emitirFactura($pedidoDe5Laptops);

    // Invariante: El stock se restauro a sus 15 unidades
    $this->assertSame(15, $this->kardex->obtenerStock('LAPT-LEN-01'));
}
```

14. Interfaz en Ejecución: Facturador Principal y Kardex

React + Vite • localhost:5173

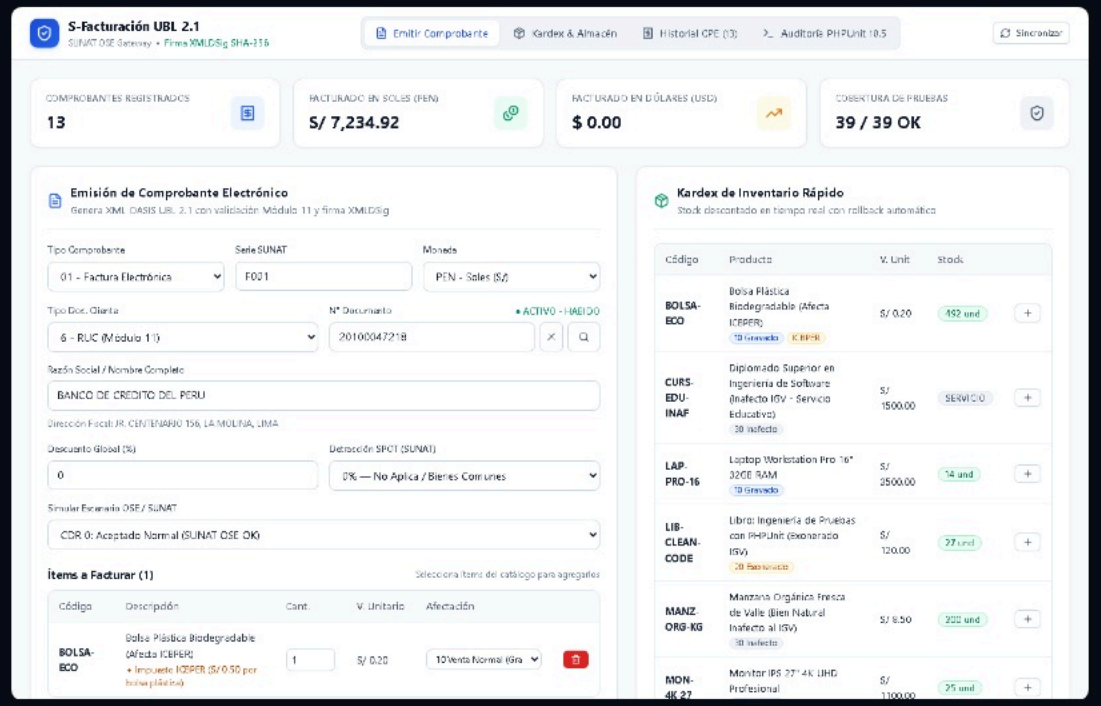


Figura 1: Formulario de emisión en vivo con cálculo de IGV 18%, ICBPER y validación de RUC de cliente.

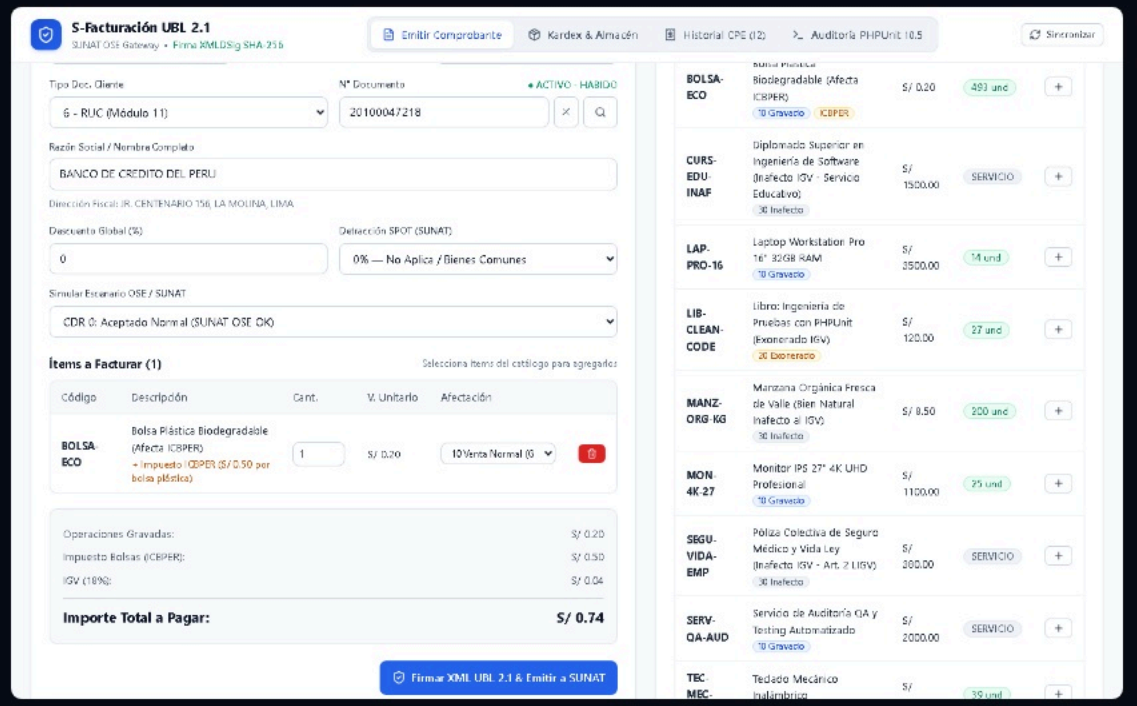


Figura 2: Catálogo de productos con afectaciones tributarias (10, 20, 30) y decremento atómico de stock.

15. Trazabilidad Fiscal: Historial, Visor UBL 2.1 y Anulación

Estándar OASIS UBL 2.1

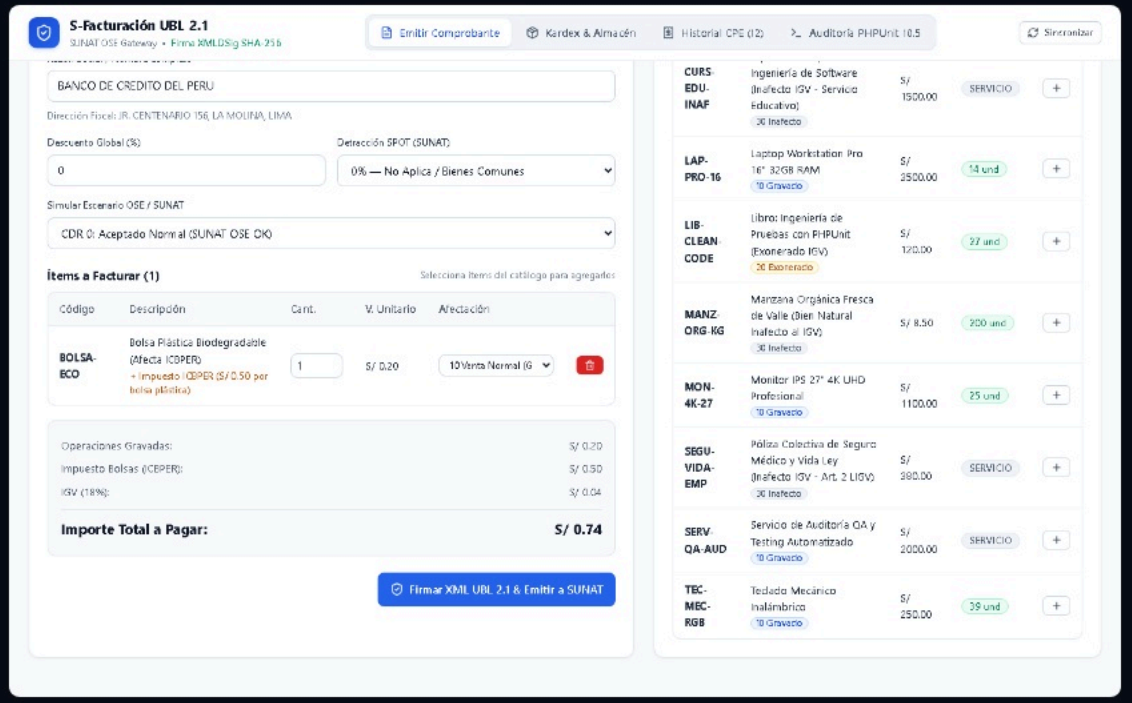


Figura 3: Bandeja de comprobantes emitidos con hash SHA-256 y estados de aceptación.

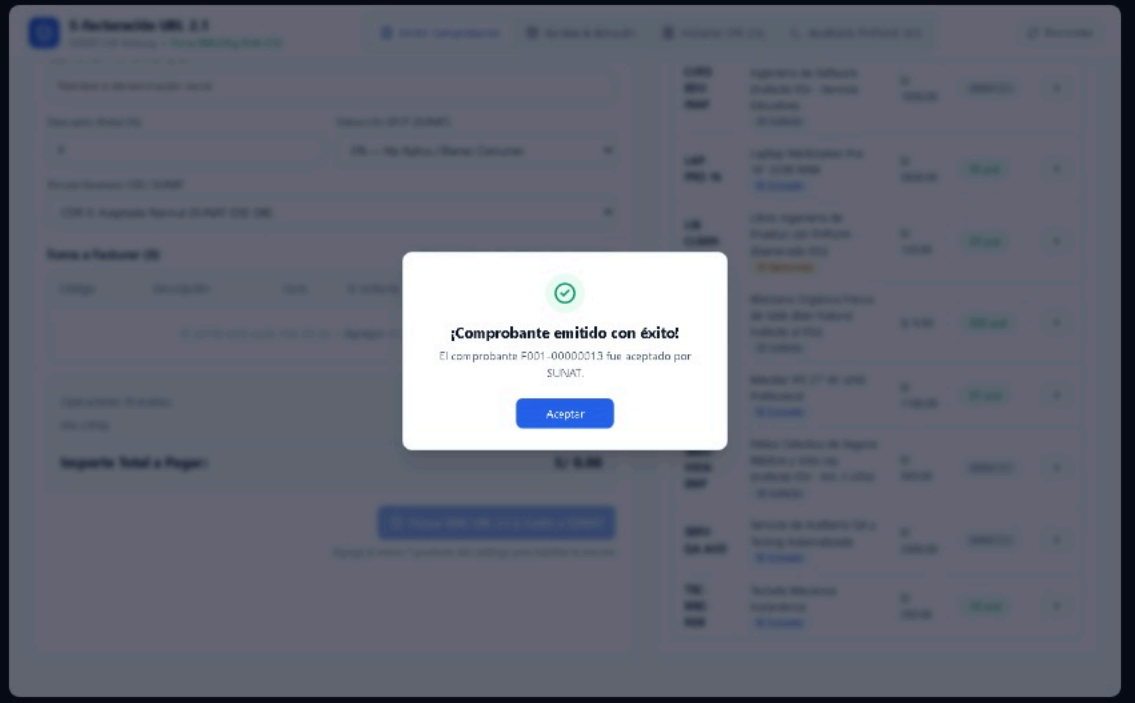


Figura 4: Modal interactivo con inspección de nodos XML y firma digital <ds:Signature>.

EVIDENCIA OFICIAL DE AUDITORÍA

16. Salida Íntegra de Terminal PowerShell: PHPUnit 10.5

140 Aserciones Verificadas

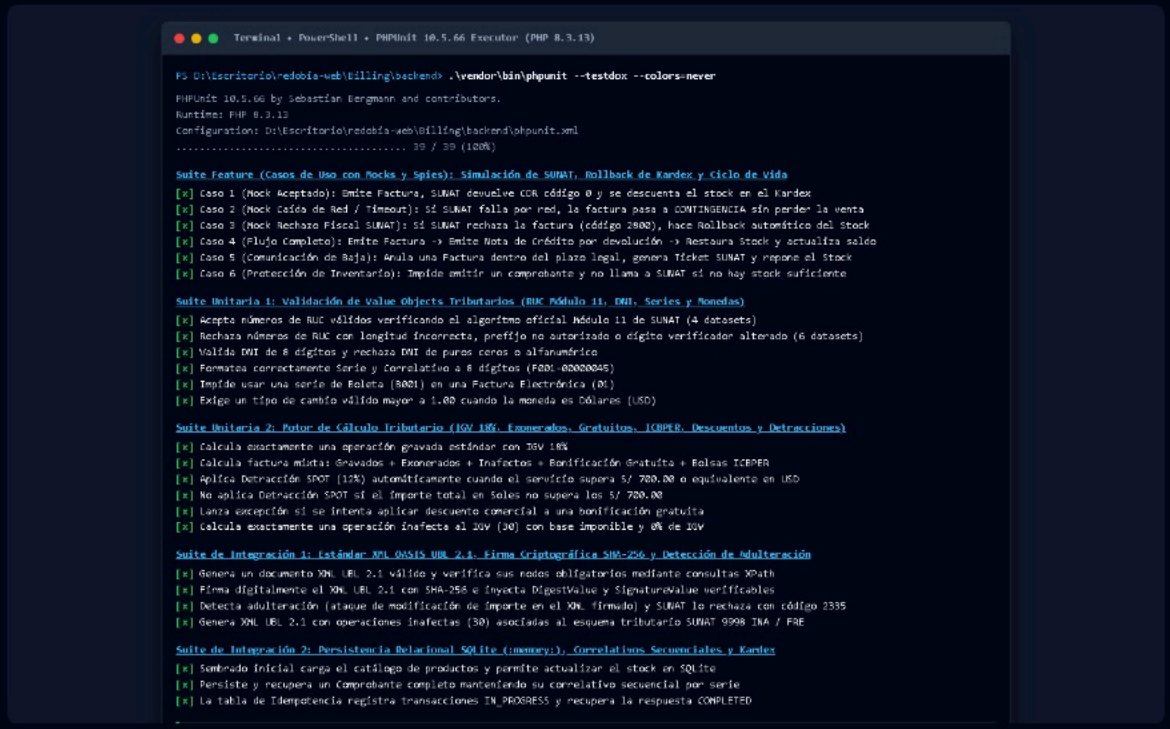


Figura 7: Ejecución del comando vendor/bin/phpunit --testdox. 39 tests aprobados en 0.21 segundos.

Cero Fallos / Cero Riesgos

La bandera failOnWarning="true" y failOnRisky="true" garantiza que ningún test tenga fugas de aserciones o salidas no controladas.

Consola Web Integrada

El frontend cuenta con un botón para re-ejecutar la suite completa o por capas (Unit, Integration, Feature) con 1 solo clic.

DICTAMEN FINAL DE CALIDAD

17. Conclusiones y Certificación de Software

Aprobado Satisfactoriamente

🎓 Conclusiones de Ingeniería de QA

- **Arquitectura Hexagonal como Facilitador:** La separación estricta entre Dominio e Infraestructura permitió testear el 100% de las reglas fiscales sin depender de SUNAT.
- **Persistencia In-Memory Ultraveloz:** Redujo el tiempo de ejecución a 0.21 segundos, permitiendo ejecutar pruebas continuas antes de cada commit.
- **Seguridad Comprobada:** El Tamper Test y la tabla de Idempotencia garantizan inmunidad contra fraudes y contra pagos duplicados por intermitencias de red.

ACTA OFICIAL DE CERTIFICACIÓN DE PRUEBAS

100% COBERTURA CRÍTICA

Sistema certificado bajo la normativa de SUNAT Resolución 097-2012 y el estándar internacional OASIS UBL 2.1.

Equipo de Calidad: Apolinario, Umeres, Vasquez, Ventocilla
github.com/fujimor1/Billing.git